

Practical No. 3

Aim :- To practice advaced SQL queries using filters (WHERE), Sorting (ORDER BY), aggregation (GROUP BY), & nested subqueries in a student Course management System.

Theory :-

① Filters (WHERE clause)

- Used to retrieve specific rows from a table based on conditions.
- Operators : = , < , > , <= , >= , != , LIKE , IN , BETWEEN.
- e.g; find students above age 20.

② Sorting (ORDER BY clause)

- Arrange query results in ascending (ASC) or descending (DESC) order.
- e.g; list courses alphabetically or by number of credits.

③ Aggregation (GROUP BY clause)

- Groups rows with the same column values & applies aggregate functions like: COUNT(), AVG(), SUM(), MAX(), MIN().
- e.g; Count number of students enrolled per course.

④ Nested Subqueries

- A query inside another query.
- Used for complex conditions where results of one query determine another query.
- e.g; find students enrolled in courses with more


```
-- DBMS Practical: Views and Indexing
-- Student Course Management System

CREATE DATABASE IF NOT EXISTS StudentDB;
USE StudentDB;

CREATE TABLE IF NOT EXISTS Students (
    Student_id INT PRIMARY KEY,
    Name VARCHAR(50),
    Age INT,
    Department VARCHAR(50)
);

CREATE TABLE IF NOT EXISTS Courses (
    Course_id INT PRIMARY KEY,
    Course_name VARCHAR(50),
    Credits INT
);

CREATE TABLE IF NOT EXISTS Enrollment (
    Enrollment_id INT PRIMARY KEY,
    Student_id INT,
    Course_id INT,
    Grade CHAR(2),
    FOREIGN KEY (student_id) REFERENCES Students(student_id),
    FOREIGN KEY (course_id) REFERENCES Courses(course_id)
);

INSERT INTO Students VALUES
(1,'Alice',20,'CS'),
(2,'Bob',21,'EE'),
(3,'Charlie',22,'ME');

INSERT INTO Courses VALUES
(101,'DBMS',4),
(102,'OS',3),
(103,'Networks',3);
```

```
INSERT INTO Enrollment VALUES
(1,1,101,'A'),
(2,1,102,'B'),
(3,2,101,'B'),
(4,3,103,'A');

CREATE OR REPLACE VIEW StudentCourseView AS
SELECT s.student_id, s.name, s.department, c.course_name, e.grade
FROM Students s
JOIN Enrollment e ON s.student_id = e.student_id
JOIN Courses c ON e.course_id = c.course_id;
SELECT * FROM StudentCourseView;

-- Update Alice's grade in DBMS course
UPDATE StudentCourseView
SET grade='A+'
WHERE name='Alice' AND course_name='DBMS';
SELECT * FROM StudentCourseView;

-- Delete Charlie's enrollment in Networks course
DELETE FROM StudentCourseView
WHERE name='Charlie' AND course_name='Networks';
SELECT * FROM StudentCourseView;

-- Index on student name
CREATE INDEX idx_student_name ON Students(name);

CREATE INDEX idx_course_name ON Courses(course_name);

CREATE INDEX idx_enroll_student_course ON Enrollment(student_id, course_id);

-- Before and after indexing (MySQL will use indexes automatically if available)
EXPLAIN SELECT * FROM Students WHERE name='Alice';
EXPLAIN SELECT * FROM Courses WHERE course_name='DBMS';
EXPLAIN SELECT * FROM Enrollment WHERE student_id=1 AND course_id=101;

-- Display student details with courses and grades after update/delete
SELECT * FROM StudentCourseView;
```

Result Grid



Filter Rows:

Edit:



Export/Import:



Wrap Cell Content:



	student_id	course_id	grade
▶	1	101	A
	1	102	B
	2	101	B
	3	103	A
	4	101	B
	4	104	A
•	NULL	NULL	NULL

than 3 credits.

Conclusion

- Filters with ~~WE~~ WHERE allow targeted data retrieval.
- Sorting using ORDER BY makes data analysis easier.
- Aggregation with GROUP BY provides summarized information.
- Nested subqueries allow complex conditional queries.
- Lab demonstrates advanced SQL querying techniques for efficient reporting in a student course management system.